Introduction to Artificial Intelligence

Chapter 6 - Part III: Uncertainty and Utilities

Namyup Kim

Soongsil University

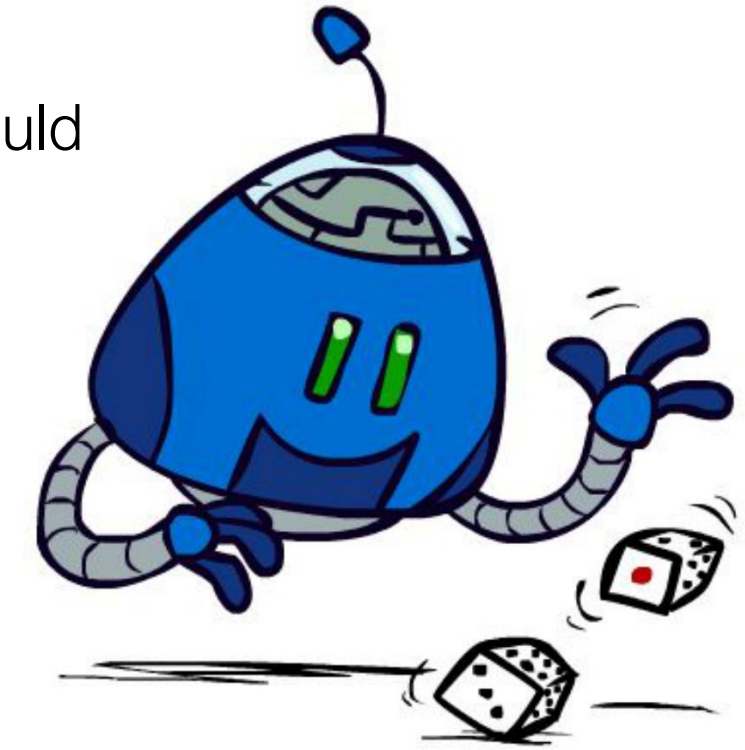


Source: UC Berkeley CS188, slides by Dan Klein and Pieter Abbeel (ai.berkeley.edu)

1. Uncertain Outcomes

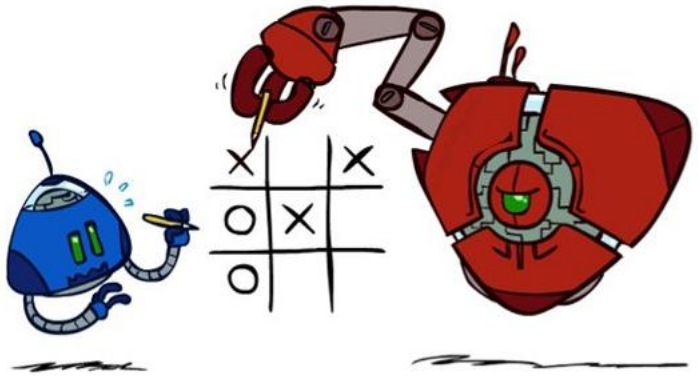
What happens when the other player is not trying to beat you

- **Worst case (MIN case)** vs **average case**, on the same tree
- **Expectimax**: the algorithm you get when **MIN** becomes an **average, not a selection.**
- Why expectimax **cannot be pruned** the way minimax could
- Depth limits again, now estimating an average

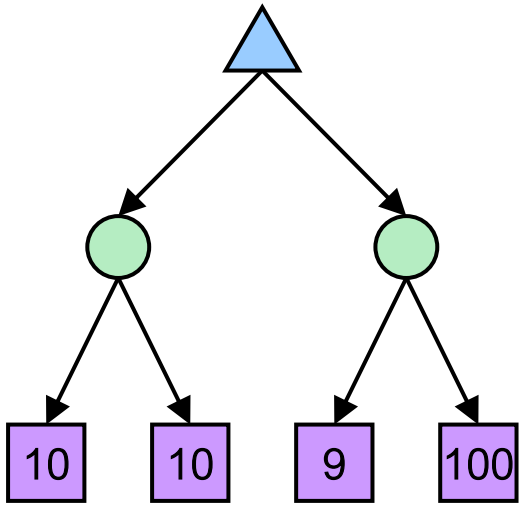


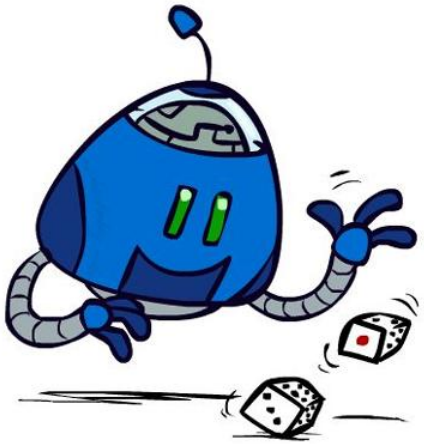
Worst-Case vs. Average Case

- The same tree, read two ways.
  - Against a minimizer (**mastermind**): left gives 10, right gives 9. The 100 is not yours.
  - Against a chance node ◯ (**rolling dice; averaging**): right is (9 + 100) / 2 = 54.5, so right wins.
- Same numbers, opposite answer. What changed is your model of that layer
- Is that leaf 100, or a billion? Under min and max it made no difference



Mastermind



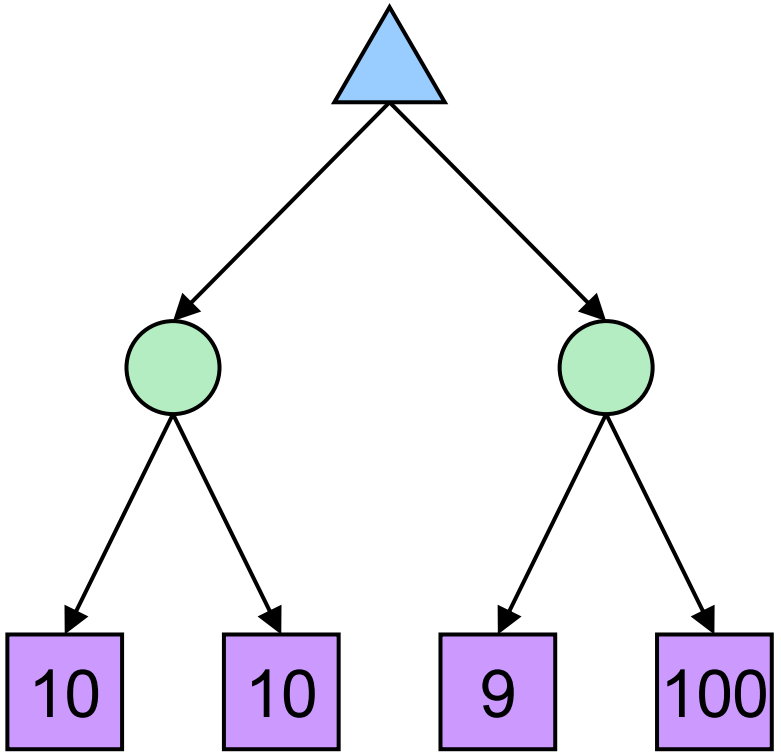


Rolling dice

Idea: Uncertain outcomes controlled by chance, not by an adversary!

Expectimax Search

- Why would we not know what an action does? Three sources:
  - Explicit randomness: rolling dice in Monopoly
  - Unpredictable opponents: random ghosts in PacMan
  - Actions failure: when moving a robot, a wheel slips.
- So, values should reflect the **average case (expectimax)** , not the worst case (minimax)
- ***Expectimax search***: compute the average score under optimal play
  - Max nodes as in minimax search
  - Chance nodes are like min nodes but the outcome is uncertain
  - Calculate their expected utilities
    - I.e., take weighted average (expectation) of children
- Later, we'll learn how to formalize the underlying uncertain-result problems as Markov Decision Processes, which is Chapter 7.



Expectimax Pseudocode

```
def value(state):
    if the state is a terminal state: return the state's utility
    if the next agent is MAX: return max-value(state)
    if the next agent is EXP: return exp-value(state)
```

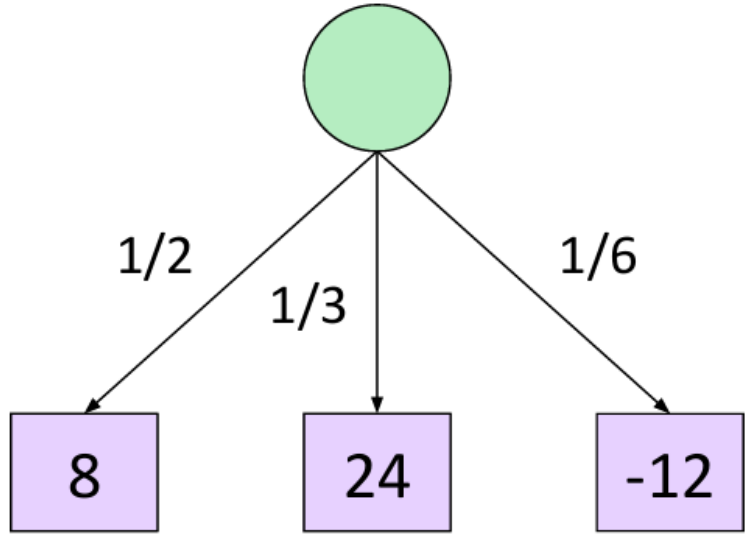
```
def max-value(state):
    initialize v = -∞
    for each successor of state:
        v = max(v, value(successor))
    return v
```

```
def exp-value(state):
    initialize v = 0
    for each successor of state:
        p = probability(successor)
        v += p * value(successor)
    return v
```

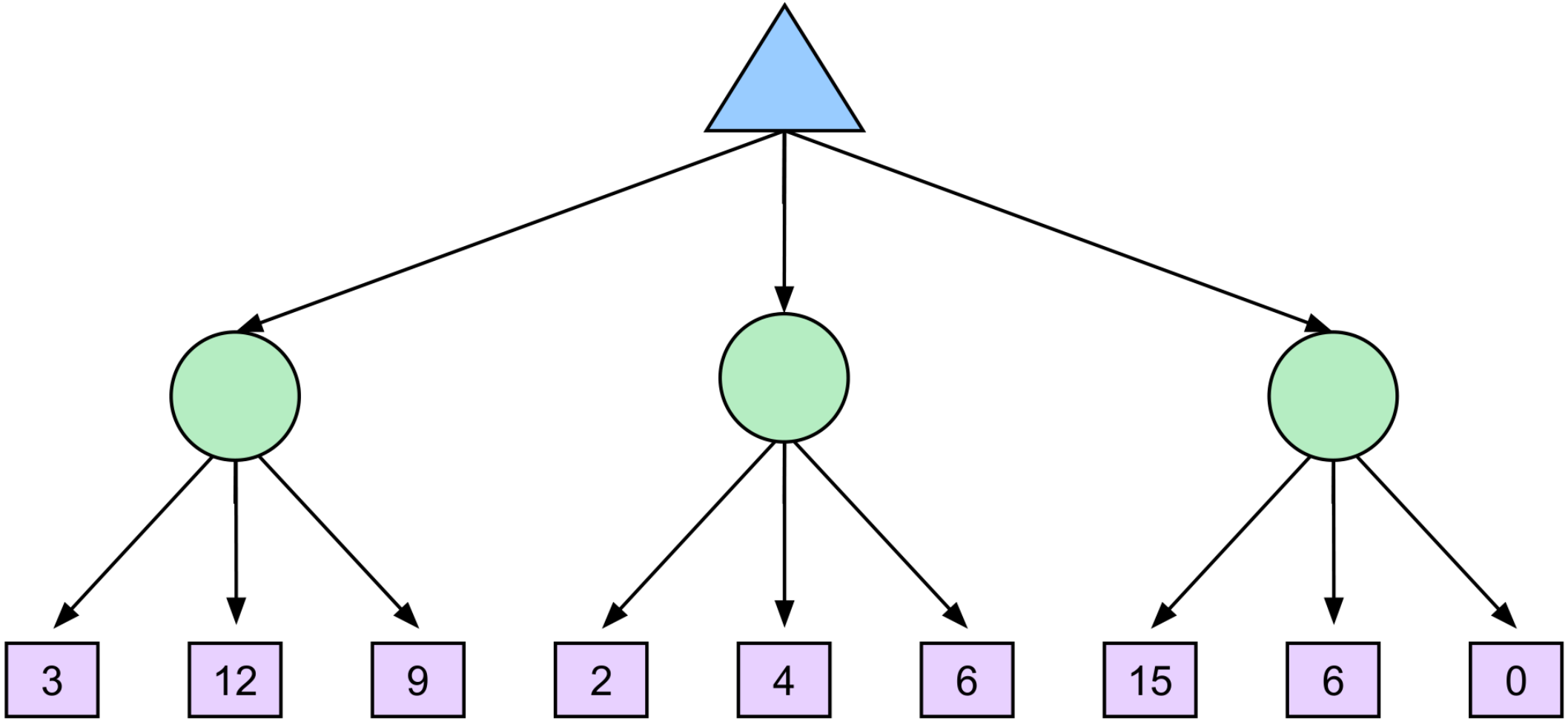
- Same dispatch as minimax; the **MIN case** is simply swapped for an **EXP case**.
- max-value starts at $-\infty$ and **rises**. exp-value starts at 0 and **accumulates**.
- That difference is why alpha-beta will not work here.

Expectimax: a Weighted Average

```
def exp-value(state):
    initialize v = 0
    for each successor of state:
        p = probability(successor)
        v += p * value(successor)
    return v
```



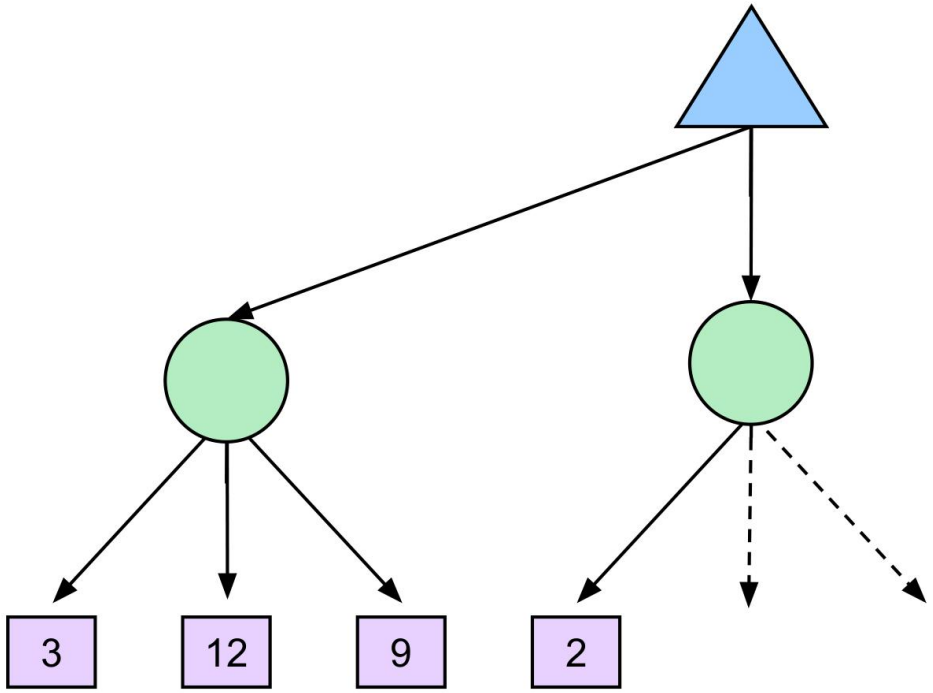
Expectimax Example

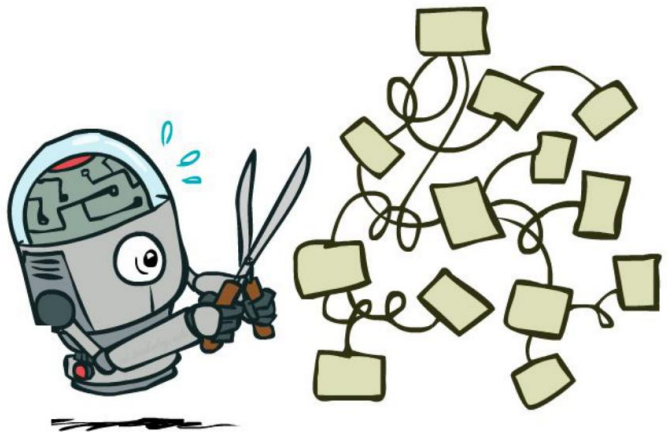


Expectimax Pruning?

- Can we prune as alpha-beta did?
- Alpha-beta relied on a MIN node **only ever falling**.
- A chance node has no such guarantee.

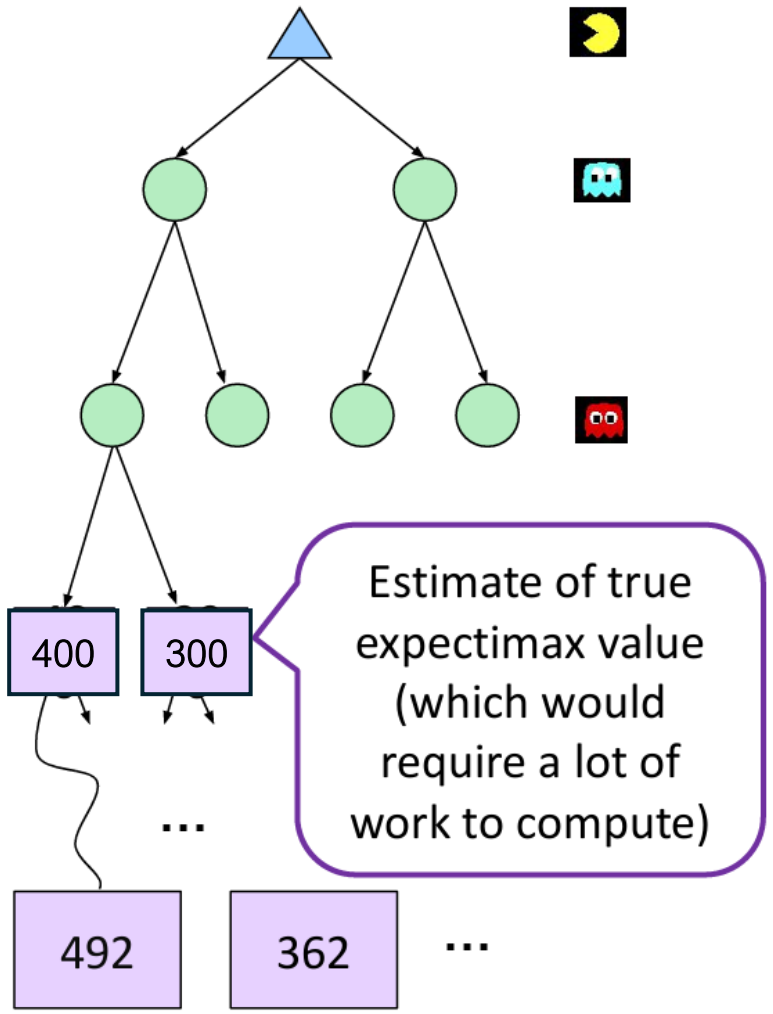
- Pruning come back only with **bounds on the magnitudes (all utility in 0 to 10)**, or **tiny probabilities** (0.999 you have already).





Depth-Limited Expectimax

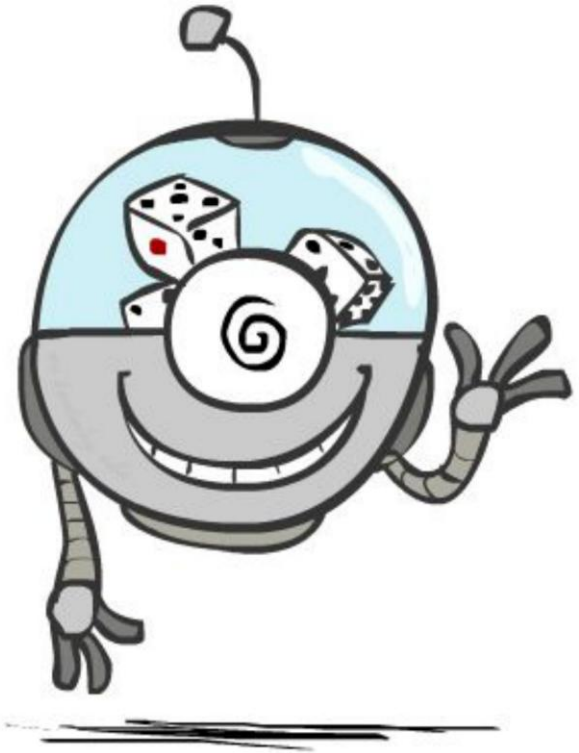
- Still too big, so we still stop early and still need an evaluation function
- It now estimates the expectimax value, not the minimax value
  - For Pac-Man: the average score from here on average at ghost nodes if they are random; minimize if they are adversarial
- You could learn it from played games, but ***you want the value under good play***.



2. Probabilities

The machinery the average needs, in one section

- Random variables and distributions
- **Expectation**: the weighted average, and why it is usually not one of the values
- Where the probabilities come from, and what they actually mean
- A worked modelling question of the kind that turns up in exams



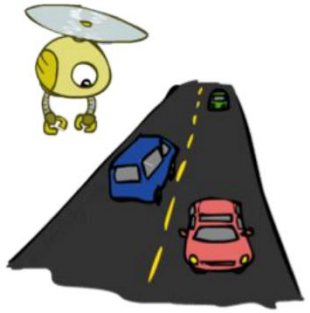
Reminder: Probabilities

- A ***random variable*** represents an event whose outcome is unknown.
- A ***probability distribution*** is an assignment of weights to outcomes.



0.25

- Example: Traffic on freeway
  - Random variable: T = whether there's traffic
  - Outcomes: T in {none, light, heavy}
  - Distribution: P(T=none) = 0.25, P(T=light) = 0.50, P(T=heavy) = 0.25



0.50

- Some laws of probability (more later):
  - Probabilities are always non-negative
  - Probabilities over all possible outcomes sum to one



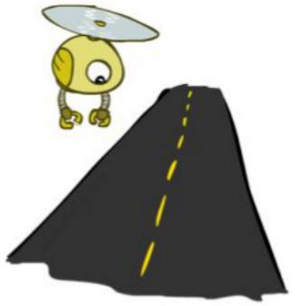
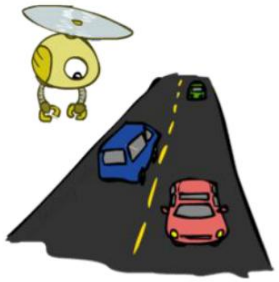
0.25

- As we get more evidence, probabilities may change:
  - P(T=heavy) = 0.25, P(T=heavy | Hour=8am) = 0.60
  - We'll talk about methods for reasoning and updating probabilities later

Reminder: Expectations

- The expected value is the average, weighted by the distribution
- How long to get to the airport?
  - (0.25)(20) + (0.50)(30) + (0.25)(60) = 35 min
- You will never experience a 35 minute trip. It is not one of the three
- It is the number to plan with, and what a chance node computes

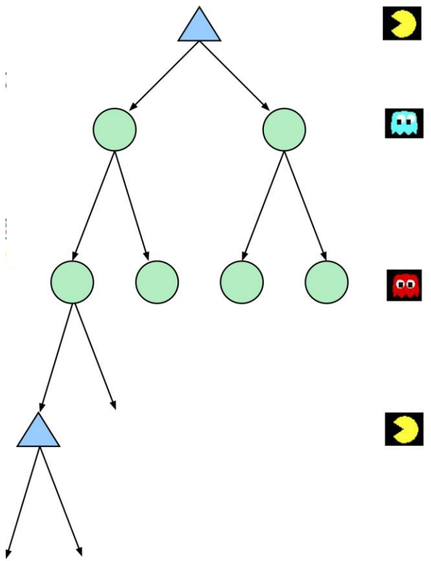
Time:	20 min	+	30 min	+	60 min		
	x		x		x		
Probability:	0.25		0.50		0.25		35 min


What Probabilities to Use?

- In expectimax search, we have ***a probabilistic model*** of how the opponent (or environment) will behave in any state.
  - Model could be a simple uniform distribution: rolling a die
  - Model could be sophisticated and require a great deal of computation
  - We have a chance node for any outcome out of our control: opponent or environment [1]
  - The model might say that adversarial actions are likely!

- For now, the probabilities arrive attached magically. Where they come from is later.



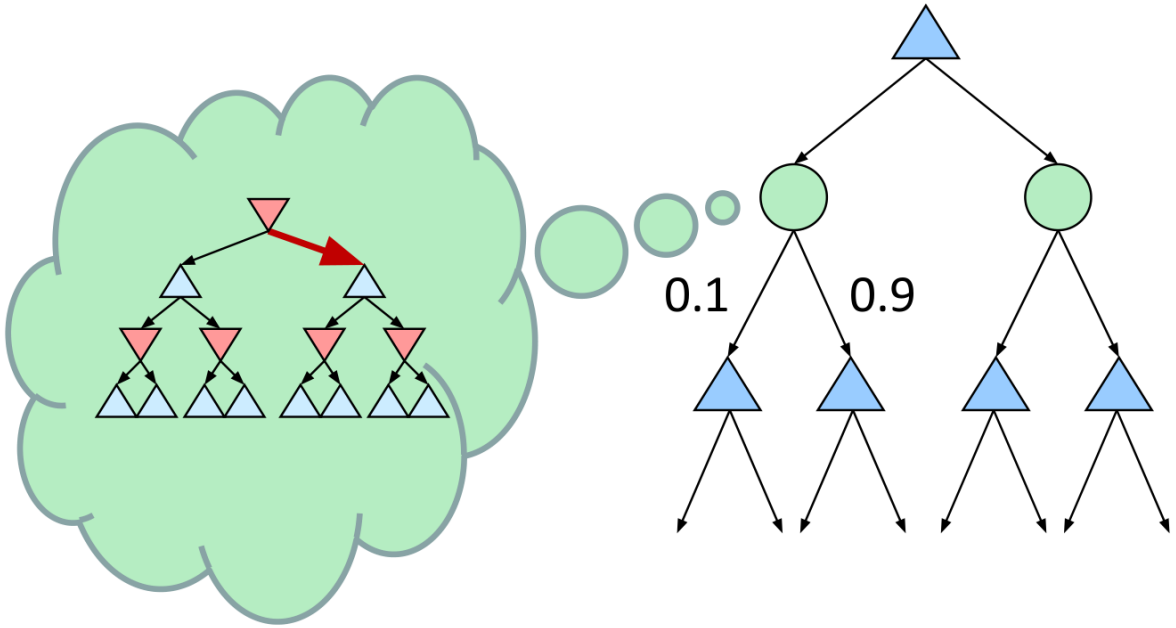
A probabilistic belief about another agent does not mean that the agent is flipping coins. [2]

[1] 우리가 **통제할 수 없는 결과**마다 **찬스 노드(chance node)**를 둔다: **상대(opponent)** 든 **환경(environment)** 이든
- 이유: 내가 고를 수 없는 결과는 확률 모델로 평균(기댓값)을 내야 하기 때문
- 대조: MAX 노드 = 내가 고름(최댓값) ↔ 찬스 노드 = 확률로 정해짐(가중 평균)
- 예: 주사위 굴리기(환경), 무작위로 움직이는 유령(상대) → 둘 다 찬스 노드

[2] 상대에 대해 **확률적 믿음(probabilistic belief)**을 갖는다고 해서, 상대가 **실제로 동전을 던지듯 무작위로 행동한다는 뜻은 아니다**
- 이유: 확률은 상대의 성질이 아니라 **상대를 잘 모르는 나의 불확실성**을 나타냄
- 대조: 주사위처럼 진짜로 무작위인 환경 ↔ 정해진 전략이 있지만 내가 모르는 상대
- 예: p14처럼 상대가 깊이 2 미니맥스를 80% 쓴다고 믿으면, 상대는 계산대로 움직이지만 나는 기댓값(expectimax)으로 계산

Quiz: Informed Probabilities

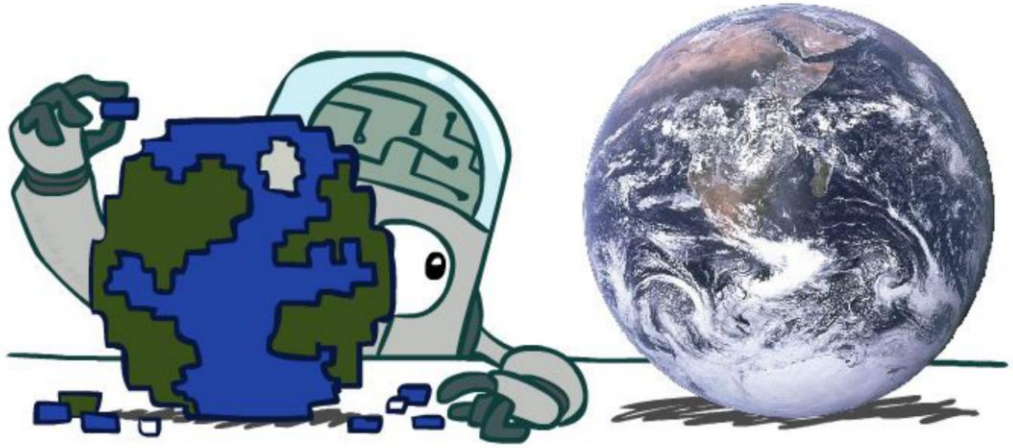
- Let's say you know that your opponent is actually running a depth 2 minimax, using the result 80% of the time, and moving randomly otherwise
- Question: What tree search should you use?
- Answer: **Expectimax**.
  - Minimax is the special case where the opponent is optimal with probability 1
  - So, expectimax is the generalization, not the alternative.
- The cost: each chance node needs you to simulate your opponent
  - Worse if they are simulating you at the same time



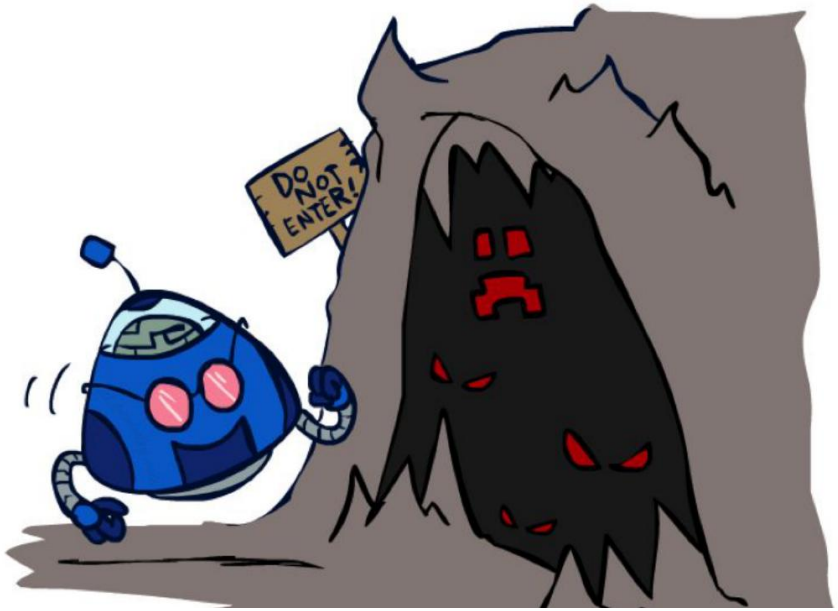
3. Modeling Assumptions

Getting the algorithm right is not the same as getting the world right

- **Dangerous optimism** and **dangerous pessimism**, side by side
- The 2 x 2 experiment: two Pac-Man agents against two kinds of ghost
- Why the two ways of being wrong **do not cost the same**



The Dangers of Optimism and Pessimism



Dangerous Optimism

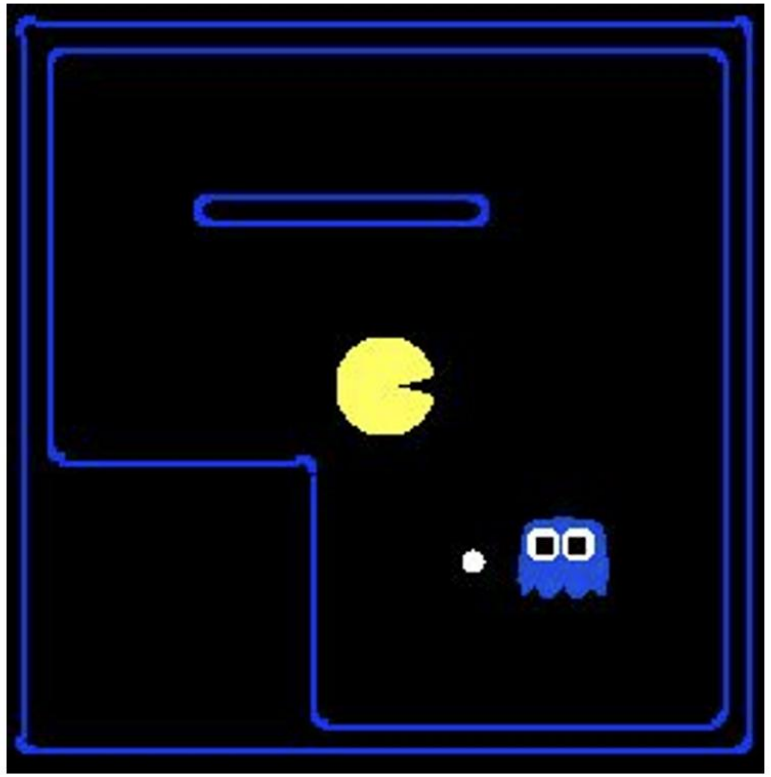
- Assuming chance when the world is **adversarial**
- You treat the bad outcomes as unlucky, so you walk into them.
- Worse: an adversary will **arrange** exactly the situations you dismissed as unlikely



Dangerous Pessimism

- Assuming the worst case when it is **not likely**
- You run **away from everything** and give up value for safety.
- The robot that is terrified of a rabbit

Assumptions vs. Reality



	Adversarial Ghost	Random Ghost
Minimax Pacman	Won 5/5 Avg. score **483**	Won 5/5 Avg. score **493**
Expectimax Pacman	Won 1/5 Avg. score **-303**	Won 5/5 Avg. score **503**

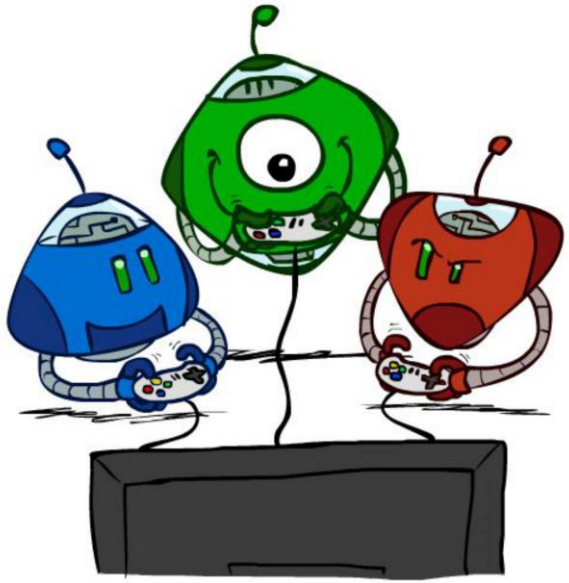
Results from playing 5 games

Pacman used depth 4 search with an eval function that avoids trouble.
Ghost used depth 2 search with an eval function that seeks Pacman.

4. Other Game Types

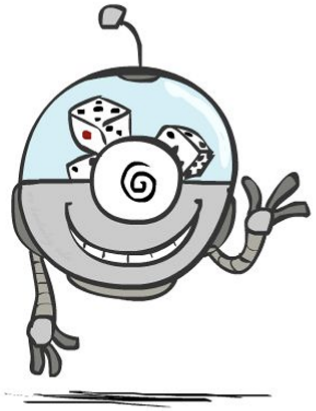
Lifting the assumptions, one at a time, and finding that everything still works

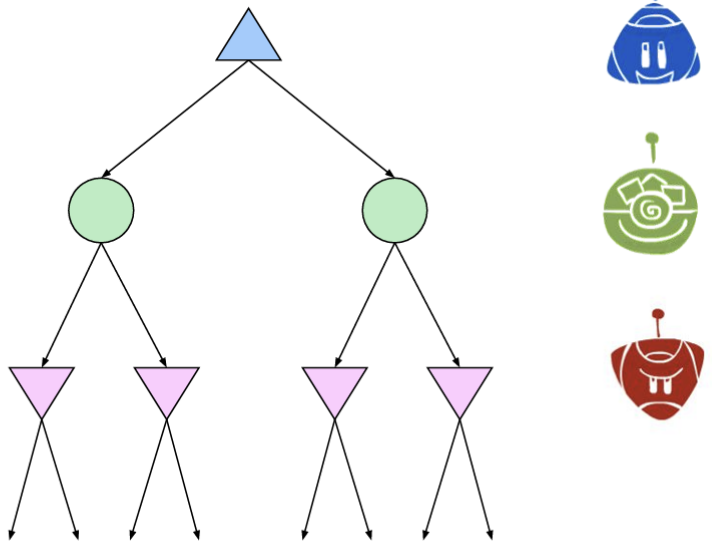
- **Mixed layers**: max, min and chance in the same tree
- **Monopoly**, and why a wide chance layer changes the arithmetic
- **More than two players**, and games that are not zero-sum
- Utility **tuples**, and where cooperation comes from



Mixed Layer Types

- There is no rule you putting max, min and chance layers in one tree
- The name is ***expectiminimax***,
- The environment is an extra player that is not adversarial
- Then each node just does what its own layer calls for





Example: Monopoly

- Expectiminimax
  - Environment is an extra “random agent” player: the dice roll that comes after each min/max agent’s move
  - MAX decides (buy / build / pass) → dice decide where the opponent lands → MIN decides → dice again …
  - Each node computes the appropriate combination of its children



Multi-Agent Utilities

- What if the game is ***not zero-sum***, or has ***more than two players***?
- Terminals carry a utility ***tuple***, one entry per player. Node values too
- Each player maximizes its own component
- There is **no minimizer** anywhere in this tree. Everyone maximizes
- Zero-sum was the special case where one number was enough
- **Cooperation** and **competition** appear by themselves dynamically

